

The harness is the signal

Plicara Research · 2026-09-02 ·

Every capable AI system has a split personality. A probabilistic model makes judgements about what to do next; the surrounding runtime turns those judgements into actions that can be bounded, inspected and recovered. This essay calls that surrounding runtime the *harness*: the tools, execution state, permissions, approvals, limits, evaluation and observability around a model.

We do not yet know which parts of the current AI stack will become durable abstractions. We do know that many of the engineering problems now appearing around language models are familiar: coordinating work, managing state, granting permissions, handling failure, observing behaviour and recovering from partial progress.

That does not mean AI systems are merely distributed systems with a new interface. The intelligence at the centre is probabilistic, the objective is often semantic rather than numeric, and the boundary between a model's judgement and the surrounding runtime is still moving. The useful question is therefore not whether today's agent frameworks are the final answer. It is which responsibilities belong durably outside the model, and which are temporary scaffolding for imperfect models.

From prompt to runtime

The first generation of LLM applications made the prompt look like the main engineering object. In more capable systems, the prompt is only one component. A production agent also needs tools, instructions, execution state, limits, approvals, evaluation and guardrails. OpenAI's [guide to building agents](#) describes an agent as a system in which the model manages workflow execution while using external tools within defined guardrails.

That is a meaningful shift. Once a model can act across a collection of external systems, the hard problems become less about producing a good completion and more about making a sequence of actions safe, inspectable and recoverable. A failed tool call, an ambiguous result or an interrupted run is not solved by a better paragraph in a system prompt.

This is the sense in which the harness is a signal. Its growing machinery shows where the practical boundary of the model currently is. It does not prove that every piece of that machinery deserves to become a permanent layer.

The strongest case for simplicity

There is a serious counterargument: much of what is called harness engineering may be compensation for limitations that better models will remove. Anthropic's [2024 production guidance](#) is deliberately conservative: start with the simplest solution, add complexity only when it demonstrably improves outcomes, and prefer simple, composable patterns to elaborate frameworks.

That advice matters because agent architecture can create its own failure modes. More agents mean more handoffs, context boundaries, traces, retries and places for a system to become difficult to evaluate. A single model with well-defined tools can often handle work that initially appears to require an agent society. OpenAI likewise recommends maximising a single agent's capabilities and adding tools before prematurely introducing multi-agent orchestration.

So the existence of an orchestration loop is not evidence that orchestration is the durable abstraction. It may be a temporary way to make a less capable model reliable enough for a specific task. The relevant test is empirical: does the extra structure improve the outcome enough to justify its operational cost?

The model-runtime boundary

The useful distinction is not between old software and new AI. It is between responsibilities that must remain governed outside a model and techniques that may fade as models become more capable. In the table below, *structural* means likely to remain external to the model; *compensatory* means likely to shrink with better model performance; and *mixed* means the answer depends on the task and must be measured.

RESPONSIBILITY	LIKELY CHARACTER	WHY
Authentication, authorization and approvals	Structural	These are organizational and security constraints, not reasoning tasks.
Audit logs, traces and policy enforcement	Structural	External systems must be able to inspect and govern consequential actions.

RESPONSIBILITY	LIKELY CHARACTER	WHY
Checkpoints, retries and recovery	Structural	Long-running work still encounters network failures, rate limits and partial progress.
Tool interfaces and schemas	Likely structural	Models need a reliable way to discover and invoke capabilities outside themselves.
Reflection, self-critique and repeated prompting	Possibly compensatory	Stronger models may need fewer explicit loops for some tasks.
Planner or decomposer subagents	Mixed	They may isolate or specialize work, but can also compensate for weak task execution.

Authentication and audit logging are not new requirements created by agents. That is precisely the point. A model may choose an action, but an organization must still decide what it is allowed to do, preserve a record of what happened and recover from failures in systems the model does not control. The structural part of the harness is the governance and reliability contract at that boundary, not a claim that every agent framework's abstractions are permanent.

Workflow systems offer a useful analogy: durable execution separates the work a program intends to do from the mechanics that let it pause, retry, resume and survive a process failure. [Temporal's documentation](#) describes that class of problem. Distributed-systems practice adds observability, partial failure and explicit service-level objectives, concerns developed at length in Google's *[Site Reliability Engineering](#)*. Neither analogy settles the AI architecture question. They identify constraints that remain when the reasoning component is stochastic.

Standardization is a leading indicator

[Model Context Protocol](#) is more informative as a signal than as a verdict. It specifies a standard way for an LLM application to connect to external tools and data sources. Its value is not that a particular protocol must win. The important fact is that tool access, schemas, permissions and integration boundaries are being treated as system-level interfaces rather than one-off prompt conventions.

If that trend continues, the stable layer may look less like a universal agent framework and more like ordinary systems engineering around an unusually capable and nondeterministic component: interoperable interfaces, secure capability access, durable state, evaluation and observability. The model may own more of the planning; the runtime may own the contract with the rest of the world.

What would change the conclusion?

This argument should be falsifiable. The harness-is-the-signal thesis weakens if more capable models consistently eliminate orchestration complexity without sacrificing reliability, safety or cost. It strengthens if the complexity that remains converges on stable runtime responsibilities rather than endlessly changing prompt patterns.

Watch four things:

1. **Runtime complexity:** Do production systems become simpler as models improve, or do they retain durable execution, evaluation and governance layers?
2. **Where performance gains come from:** Are improvements attributable primarily to base models, or to better tools, state handling, verification and execution environments?
3. **Standardization:** Do tool and context interfaces converge across vendors and applications?
4. **Operational ownership:** Which concerns remain owned by platform, security and reliability teams even when model behaviour improves?

The conclusion is not that AI has rediscovered operating systems, distributed systems, workflow engines or control theory. It is that those fields give us a disciplined way to separate a structural requirement from a compensating trick. We do not yet know which pieces make up the final picture of AI. We do know the pieces on the table, and we can test which ones survive contact with better models.